4주차 공부 내용 정리

교재 5장 | 반응, 계획, 도구 사용, 멀티에이전트, CoT 프롬프팅 패턴 | 팀 알아서할게

4주차에는 교재 5장을 읽고 AI Agent를 설계할 때 자주 쓰는 패턴을 정리했다. 3주차 회고에서 정한 대로 패턴마다 흐름도를 붙여서 구조를 비교했다.

01 설계 패턴 개요

설계 패턴은 LLM을 언제 몇 번 부르고 그 결과를 어디로 넘길지 정하는 구조다. Andrew Ng는 2024년 DeepLearning.AI 뉴스레터에서 agentic design pattern을 Reflection, Tool Use, Planning, Multi-agent Collaboration으로 나눴는데, 교재 5장의 반응, 도구 사용, 계획, 멀티에이전트 패턴이 이 분류와 대응한다. CoT 프롬프팅은 LLM 호출 하나 안에서 추론 방식을 바꾸는 기법이라서 다른 패턴 안에 섞여 쓰인다.

패턴	하는 일	LLM 호출	주의할 점
반응	만든 결과를 스스로 평가하고 고친다.	생성 1회, 평가와 수정 반복	같은 모델이 평가하면 자기 오류를 놓치기 쉽다.
계획	목표를 단계로 나누고 순서대로 실행한다.	계획 1회, 단계마다 실행	처음 계획이 틀리면 뒤 단계가 모두 영향을 받는다.
도구 사용	외부 함수와 API를 불러 정보를 얻거나 작업을 실행한다.	도구 결과를 받을 때마다 1회	도구 결과를 검증하지 않으면 잘못된 값이 그대로 쓰인다.
멀티에이전트	역할이 다른 에이전트가 일을 나눠 맡고 결과를 주고받는다.	에이전트 수만큼 늘어난다	비용과 응답 시간이 늘고 전달 과정에서 맥락이 빠진다.
CoT	답을 내기 전에 추론 과정을 단계별로 쓴다.	1회, Self-Consistency는 여러 번	추론 과정이 그럴듯해도 답이 틀릴 수 있다.

02 반응 패턴

반응 패턴은 LLM이 만든 결과를 다시 LLM에 보여 주고 문제점을 찾게 한 뒤, 그 피드백으로 결과를 고치는 패턴이다. 사람이 초안을 쓰고 다시 읽으며 고치는 과정과 비슷하다.

생성 → 평가 → 수정 → 평가 통과 시 종료 통과하지 못하면 평가와 수정을 반복

생성과 평가 나누기

2023년 Self-Refine 논문은 모델 하나가 초안 작성, 피드백, 수정을 번갈아 맡게 했고, 코드 최적화와 글쓰기 같은 과제에서 한 번에 답한 결과보다 품질이 높았다. 평가 역할을 다른 프롬프트나 다른 모델에 맡기면 생성한 쪽이 놓친 문제를 찾기 더 쉽다.

외부 피드백 사용

외부 근거 없이 모델이 스스로 고치게 하면 맞는 답을 틀린 답으로 바꾸기도 한다는 결과가 있다(Huang et al., 2024). 그래서 코드라면 테스트 실행 결과, 사실 확인이라면 검색 결과처럼 밖에서 얻은 근거를 평가 단계에 넣는다. 3주차에서 본 Reflexion은 실패 원인을 메모리에 남겨 다음 시도에 쓰는 방식으로 반응 패턴을 여러 번의 시도로 넓혔다.

순서	단계	코드 작성 에이전트 예시
01	생성	"리스트의 최댓값을 구하는 함수를 만들어 줘"라는 요청에 초안 코드를 쓴다.
02	평가	테스트 3개를 실행하고, 빈 리스트를 넣은 테스트가 오류로 실패한 결과를 받는다.
03	수정	실패 메시지를 프롬프트에 넣어 빈 리스트일 때 `None` 을 돌려주도록 고친다.
04	종료	테스트가 모두 통과하면 최종 코드를 답한다.

03 계획 패턴

계획 패턴은 실행 전에 전체 작업 목록을 먼저 만들고 그 목록을 따라 하위 작업을 처리하는 패턴이다. 3주차에서는 계획을 에이전트의 구성요소로 봤고, 5장에서는 계획과 실행을 어떤 구조로 연결하는지를 다룬다.

목표 → 계획 수립 → 단계 실행 → 결과 확인 → 최종 답 결과가 예상과 다르면 재계획

방식	동작
Plan-and-Execute	계획 담당 LLM이 단계 목록을 만들고 실행 담당이 하나씩 처리한다. 단계가 끝날 때마다 남은 계획을 고칠 수 있다.
ReWOO	계획 단계에서 필요한 도구 호출을 모두 적어 두고 결과는 #E1, #E2 같은 변수로 받아 쓴다. 실행 중에 LLM을 다시 부르지 않아 토큰이 적게 든다(Xu et al., 2023).
HuggingGPT	LLM이 작업을 나누고 단계마다 알맞은 Hugging Face 모델을 골라 실행한 뒤 결과를 모아 답한다(Shen et al., 2023).

계획 패턴은 단계가 많고 순서가 중요한 작업에 맞다. 다음 행동이 직전 결과에 크게 좌우되는 작업은 미리 세운 계획이 자주 바뀌어서, 단계마다 판단하는 ReAct 방식이 더 낫다.

04 도구 사용 패턴

도구 사용 패턴은 LLM이 필요한 시점에 도구를 고르고, 실행 결과를 보고 다음 행동을 정하는 패턴이다. 도구를 정의하고 호출하는 function calling은 3주차에 정리했으니 여기서는 호출이 이어지는 흐름을 본다.

질문 → 생각 → 도구 호출 → 관찰 → 답 정보가 부족하면 생각부터 다시 반복

ReAct 실행 기록 예시

```
질문: 내일 부산 최고 기온은 서울보다 몇 도 높은가?
Thought: 두 도시의 내일 최고 기온을 알아야 한다.
Action: get_weather(city="부산", date="내일")
Observation: 최고 18도
Action: get_weather(city="서울", date="내일")
Observation: 최고 14도
Thought: 18 - 14 = 4
Answer: 부산이 서울보다 4도 높다.
```

병렬 호출과 오류 처리

서로 결과가 필요 없는 도구 호출은 한 번에 요청할 수 있다. 위 예시의 부산과 서울 날씨 조회가 여기에 해당하고, OpenAI와 Anthropic API는 이런 호출을 한 응답에 함께 담는 parallel tool calling을 지원한다.

Toolformer(Schick et al., 2023)는 모델이 언제 어떤 API를 부를지 스스로 학습하게 한 연구다. 실제 구현에서는 도구 설명과 입력 형식을 명확히 쓰고, 도구가 실패하면 오류 메시지를 모델에 그대로 넘겨 다시 시도하게 하는 처리를 함께 둔다.

05 멀티에이전트 패턴

멀티에이전트 패턴은 역할, 프롬프트, 도구를 다르게 준 에이전트 여러 개가 작업을 나눠 맡는 패턴이다. 에이전트 하나에 모든 도구와 지시를 넣으면 프롬프트가 길어지고 도구 선택이 부정확해지는데, 역할을 나누면 각 에이전트가 맡은 일만 처리한다.

사용자 요청 → 감독 에이전트 → 검색, 작성, 검토 에이전트 → 감독 에이전트 → 최종 답

구조	동작	예시
순차	앞 에이전트의 결과를 다음 에이전트가 받아 처리한다.	기획, 개발, 테스트 역할을 차례로 거치는 ChatDev, MetaGPT
감독자	감독 에이전트가 작업을 나눠 하위 에이전트에 맡기고 결과를 모은다.	LangGraph의 supervisor 구조
대화와 토론	에이전트들이 메시지를 주고받으며 답을 다듬거나 서로 반박한다.	AutoGen 그룹 채팅, Multi-agent Debate(Du et al., 2023)

> Anthropic은 2024년 글 Building effective agents에서 단순한 구조로 시작하고 필요할 때만 복잡도를 올리라고 권한다. 2025년에 공개한 멀티에이전트 리서치 시스템은 일반 채팅보다 토큰을 15배 정도 더 썼다고 밝혔기 때문에, 에이전트 하나로 해결되지 않는다는 것이 확인됐을 때 나누는 편이 낫다.

06 CoT 프롬프팅 패턴

Chain-of-Thought(CoT) 프롬프팅은 답만 묻지 않고 풀이 과정을 단계별로 쓰게 해서 추론 정확도를 높이는 기법이다. Wei et al.(2022)은 풀이 과정이 담긴 예시 몇 개를 프롬프트에 넣었을 때 산술과 상식 추론 문제의 정답률이 크게 오른다는

것을 보였다.

```
질문: 식당에 사과가 23개 있었다. 점심에 20개를 쓰고 6개를 더 샀다. 사과는 몇 개인가?

답만 요구한 경우: 27개
CoT로 답한 경우: 처음 23개에서 20개를 쓰면 3개가 남는다. 6개를 더 사면 3 + 6 = 9개다. 답은 9개다.
```

Wei et al.(2022) 논문의 그림 1에 나온 예시를 옮겼다.

방식	방법	출처
Few-shot CoT	풀이 과정을 포함한 예시를 질문과 함께 준다.	Wei et al., 2022
Zero-shot CoT	예시 없이 "단계별로 생각해 보자"는 문장만 붙인다.	Kojima et al., 2022
Self-Consistency	같은 질문으로 풀이를 여러 번 만들고 가장 많이 나온 답을 고른다.	Wang et al., 2022
Tree of Thoughts	풀이 단계마다 후보를 여러 개 만들고 평가해서 유망한 경로만 이어 간다.	Yao et al., 2023

CoT는 다른 패턴 안에서도 쓰인다. ReAct의 Thought 단계와 계획 패턴의 작업 분해가 모두 CoT를 행동과 결합한 형태다. OpenAI o1처럼 학습 단계에서 긴 추론을 익힌 추론 모델은 CoT를 지시하지 않아도 스스로 추론 과정을 거치기 때문에, 이런 모델에는 단계별 지시보다 목표와 조건을 명확히 주는 편이 낫다.

07 더 생각해 볼 질문

- 반응 패턴에서 평가 기준은 어떻게 정해야 하는가? 기준이 모호하면 평가 결과가 매번 달라지고 수정이 끝나지 않아서, 테스트나 체크리스트처럼 확인할 수 있는 기준이 필요하다.
- 어떤 작업에서 멀티에이전트가 단일 에이전트보다 나은가? 하위 작업을 따로 병렬 처리할 수 있는 조사 작업에서는 효과가 크고, 단계끼리 맥락을 많이 공유하는 코딩 작업에서는 이점이 적다는 분석이 있다.

참고 자료: 서지영, 랭체인 & 랭그래프로 AI 에이전트 개발하기, 길벗 / Andrew Ng, Agentic Design Patterns, DeepLearning.AI The Batch, 2024 / Madaan et al., Self-Refine, 2023 / Huang et al., Large Language Models Cannot Self-Correct Reasoning Yet, 2024 / Xu et al., ReWOO, 2023 / Shen et al., HuggingGPT, 2023 / Yao et al., ReAct, 2023 / Schick et al., Toolformer, 2023 / Du et al., Improving Factuality and Reasoning in Language Models through Multiagent Debate, 2023 / Anthropic, Building effective agents, 2024 / Anthropic, How we built our multi-agent research system, 2025 / Wei et al., Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, 2022 / Kojima et al., Large Language Models are Zero-Shot Reasoners, 2022 / Wang et al., Self-Consistency, 2022 / Yao et al., Tree of Thoughts, 2023